

CS 147: Lab 02

Due Date: Feb. 24, 2026

September 22, 2026

The main objective of this problem set is to become familiar with using the toolchain for the course project. Feel free to refer to the Verilog cheatsheet, Verilog lecture slides linked off of canvas.

Some advice: Before starting to write any Verilog, you should do the following:

1. Break down your design into sub-modules.
2. Define interfaces between these modules.
3. Draw paper and pencil schematics for these modules (these will be handed in as scanned `schematic.pdf` file(s)).
4. Then start writing Verilog.

Problem 1 (20 points)

Design a 16-bit barrel shifter in Verilog with the following interface. If you need additional information on barrel shifter design, you can consult the ALU lecture notes. Note that we call this shifter a barrel shifter because it is capable of shifting and rotating bits all the way around in any direction (like a barrel). You should **not** just write a case statement with each constant shift amount – instead think about how the hardware would be designed for a shifter, and what underlying modules (e.g., muxes) you might be able to utilize to do shifting.

Inputs:

- [15 : 0] `InBS` - 16-bit input operand value to be shifted
- [3 : 0] `ShAmt` - 4-bit amount to shift (number of bit positions to shift)
- [1 : 0] `ShiftOper` - shift type, see encoding in table below

Output:

- [15 : 0] `OutBS` - 16-bit output operand

Opcode	Operation
00	Rotate left
01	Shift left
10	Shift right arithmetic
11	Shift right logical

(Aside: you should think about if the above 4 opcodes are sufficient to represent all of the shift operations you need to implement for your project.)

Verify the design using the testbenches provided by us. You are also encouraged to write additional tests, as we may use more tests when grading your assignment. See class slides from the Verilog lecture on how to write tests.

Problem 2 [30 points]

This problem should also be done in Verilog. Design a simple 16-bit ALU. Operations to be performed are 2's Complement ADD, bitwise-OR, bitwise-XOR, bitwise-AND, and the barrel shifter unit from Problem 1. Additionally, it must have the ability to invert either of its data inputs before performing the operation and have a Cin input (to enable subtraction). Another input line also determines whether the arithmetic to be performed is signed or unsigned. Use your CLA (from homework 1) in your design, extended as needed to take things like overflow and sign into account. For all the shift and rotate operations, assume the number to shift is input InA to your ALU and the shift/rotate amount is bits [3:0] of input InB (**Note:** InA/InB may be inverted before reaching the shifter, see below – if either/both is inverted, you should shift/rotate the inverted values instead).

Opcode	Function	Result
000	rll	Rotate left
001	sl	Shift left logical
010	sra	Shift right arithmetic
011	srl	Shift right logical
100	ADD	A+B
101	AND	A & B
110	OR	A B
111	XOR	A ^ B

The external interface of the ALU should be:

Inputs

- InA[15:0], InB[15:0] - Data input lines InA and InB (16 bits each).
- Cin - A carry-in for the LSB of the adder.
- Oper(2:0) – A 3-bit opcode that determines which operation should be performed and outputted. The opcodes are shown in the table above.

- **invA** - An invert-A input that causes the A input to be inverted before the operation is performed. **invA** is active high, which means it inverts A when **invA** is 1.
- **invB** - An invert-B input (also active high) that causes the **invB** input to be inverted before the operation is performed.
- **sign** - A flag to indicate whether signed-or-unsigned arithmetic should be performed in the ADD function on the data lines (this also affects the **Of1** output). The sign input is active high for signed arithmetic and active low for unsigned arithmetic.

Outputs

- **Out(15:0)** – Output data from your ALU (16 bits).
- **Of1** - This should be high (1) if an overflow occurred (1 bit).
- **Zero** - This should be high if the result is exactly zero (1 bit).

Other assumptions:

- You can assume 2's complement numbers.
- In case of logic functions, **Of1** is not asserted (i.e., kept logic low).

The top-level module definitions and a testbench are included in the download. **You must not change these top-level module names.**

Simulate and verify your design using the supplied testbench or create one yourself to test any of your submodules. You must reuse the barrel shifter unit designed in Problem 1. **If/when you use files from hw2_1 in Problem 2, you must copy the files over from Problem 1 – you should not assume they are in hw2_2's folder by default.**

In addition to the Verilog, you should also turn in schematics for each of your components. The schematics may be handwritten or computer generated but must be legible if they are handwritten. The schematic files should be named **schematic.pdf** and placed in the corresponding problems subdirectory (e.g., hw2_1/schematic.pdf and hw2_2/schematic.pdf). Although the schematics may seem simple for some of these components, as your project gets bigger and bigger, you'll find that drawing schematics of each component and the bigger picture will make your task significantly easier.

Testing Instructions

To run the testbench programs that we have distributed for this lab, use:

- `./run test <homework> [problem]`
- Replace **<homework>** with the assignment name (e.g., hw1).

- The [problem] argument is optional. If omitted, all tests for the selected homework assignment will be run.
- For more detailed output, add `-v` (e.g., `./run test -v hw2 hw2_1`).
 - **Warning:** `-v` output can be extremely long in some contexts.

Verilog Rules

Read the Verilog rules document at `assignments/tools/verilog_rules/verilog_rules.pdf` and adhere to the conventions. Rules adherence will be checked when you run `./run test`.

Submission Instructions

To build a submission for grading, run:

- `./run build_submission <assignment>`
- This will automatically run `./run test <assignment>`, then run the generated submission through the integrated autograder.
- A submission ZIP is created in `generated_turnins/<assignment>/`.
- If you run `build_submission` multiple times for the same assignment, the generated zip files will be prepended with a number. The most recent submission should be the item with the highest number.
- **IMPORTANT:** Running `./run build_submission <assignment>` **DOES NOT** actively submit your assignment, it only generates the file you will submit. This file must be manually uploaded to Gradescope.